FACULITY OF COMPUTING TECHNOLOGY

Department of *computer science*

Project report: – *0/1 Knapsack: Investment Portfolio Optimization*

Group Members:

1. Elias araya ----------Aku1601720
2. Mulu g/medhin ----------Aku1602465
3. Arsema birhane ----------Aku1602222

***Submitted to**: - Ekubazgi G. (MSc.)*
March 30, 2026
Aksum, Ethiopia

Contents

1. Introduction ii
1.1. The 0/1 Knapsack Problem ii
1.2. Real-World Connection: Investment Portfolio Optimization ii
1.3. Key Terms ii
2. Algorithm Design ii
2.1. Why Dynamic Programming? ii
2.2. Recurrence Relation 1
2.3. Pseudocode 1
2.4. Algorithm Comparison 1
3. Implementation Details 1
3.1. Data Structures 1
3.2. CSV Dataset Structure 1
3.3. Dataset Design — Three Tiers 2
3.4. Key Functions 2
4. Full Working Code 2
4.1. Core DP Loop 2
4.2. Space-Optimised 1D DP 2
5. Experimentation & Results 3
5.1. Test Configuration 3
5.2. Named Test Cases - All Brute-Force Verified 3
5.3. Full Dataset Results 3
6. Greedy Approach Comparison 3
6.1. Why Greedy Fails for 0/1 Knapsack 3
6.2. Full Dataset Comparison 4
7. Complexity Analysis 4
7.1. Time Complexity 4
7.2. Space Complexity 4
7.3. Measured Runtimes (actual run) 4
8. Screenshots Summary 5
9. Improvements & Extensions 5
10. Conclusion 5

1. Introduction

1.1. The 0/1 Knapsack Problem

The **0/1 Knapsack Problem** is a classic combinatorial optimization problem. Given a set of *n* items each with a **weight** and a **value**, and a knapsack with fixed **capacity W**, the goal is to select a subset such that:

- Total weight does **not exceed** capacity W
- Total value is **maximized**
- Each item is either fully included (1) or fully excluded (0) — no fractions

This binary constraint makes it NP-hard in the general case and is what separates it from the Fractional Knapsack problem.

1.2. Real-World Connection: Investment Portfolio Optimization

Knapsack Term	Investment Term	Description
Weight	Cost	Capital required to enter the investment
Value	Expected Return	Projected profit over a fixed horizon
Capacity	Total Budget	Maximum capital the investor can deploy
Item	Investment Opportunity	Stock, bond, real estate, crypto, etc.

An investor cannot partially buy a house or take half a bond position — each opportunity is either fully taken or skipped. That is exactly the 0/1 constraint.

1.3. Key Terms

- **Cost (weight):** Capital required, measured in $1,000 units
- **Return (value):** Expected profit from the investment
- **Budget (W):** Total available capital — $500K in this project
- **Portfolio:** The selected subset that maximizes return within budget

2. Algorithm Design

2.1. Why Dynamic Programming?

Brute force checks all 2^n subsets. For n=120 that is $\approx 1.3 \times 10^{36}$ combinations — impossible. DP works because the problem has two key properties:

① **Overlapping Sub problems** the best portfolio for budget *w* using items 1...i reuses answers already computed for smaller budgets. These are stored in a table instead of recomputed.

② **Optimal Substructure** If item *i* is in the optimal solution, the remaining items form an optimal solution for budget W − cost[i] using items 1...i−1.

2.2. Recurrence Relation

```
dp[i][w] = 0                                if i=0 or w=0
dp[i][w] = dp[i-1][w]                       if cost[i] > w
dp[i][w] = max( dp[i-1][w],  dp[i-1][w - cost[i]] + return[i] ) otherwise
```

2.3. Pseudocode

```
FUNCTION KnapsackDP(costs[], returns[], n, W):
    CREATE dp[0..n][0..W] = all zeros
    FOR i FROM 1 TO n:
        FOR w FROM 0 TO W:
            skip ← dp[i-1][w]
            take ← 0
            IF costs[i] ≤ w:
                take ← dp[i-1][w - costs[i]] + returns[i]
            dp[i][w] ← MAX(skip, take)
    RETURN dp[n][W]
FUNCTION Traceback(dp[][], costs[], n, W):
    selected ← []
    w ← W
    FOR i FROM n DOWN TO 1:
        IF dp[i][w] ≠ dp[i-1][w]:
            APPEND (i-1) TO selected
            w ← w - costs[i]
    RETURN REVERSE(selected)
```

2.4. Algorithm Comparison

Approach	Time Complexity	Optimal?	Notes
Brute Force	$O(2^n)$	Yes	Infeasible for $n > 25$
Greedy	$O(n \log n)$	No	Fails on 0/1 constraint
Dynamic Prog.	$O(n \times W)$	Yes	Pseudo-polynomial, practical

3. Implementation Details

3.1. Data Structures

```
ids[]     : list[str]   – Investment_ID labels
names[]   : list[str]   – human-readable names
costs[]   : list[int]   – investment costs (weights)
returns[] : list[float] – expected returns (values)
dp[][]    : 2D list     – DP table, size (n+1) × (W+1)
```

3.2. CSV Dataset Structure

Column	Type	Description
Investment_ID	string	Unique ID (e.g. INV_001)
Name	string	Globally known name (e.g. Apple)
Cost	int	Capital required ($1,000 units)

Return	float	Expected return ($1,000 units)

3.3. Dataset Design — Three Tiers

The 120 investments are structured in three tiers to expose the greedy–DP gap:

Tier	Count	Cost Range	Return Range	Purpose
A	40	$5K – $15K	$30K – $60K	Greedy bait (high ratio)
B	40	$20K – $50K	$80K – $140K	DP sweet spot (high absolute)
C	40	$10K – $90K	$5K – $100K	Realistic noise

Items are shuffled after generation so the algorithm cannot exploit ordering.

3.4. Key Functions

Function	Purpose
`load_investments`	Reads CSV into parallel arrays
`knapsack_dp_2d`	Builds full 2D DP table, returns max return + table
`traceback`	Walks back through DP table to find selected items
`knapsack_dp_1d`	Space-optimised 1D DP — O(W) space, no trace back
`knapsack_greedy`	Greedy selection by return/cost ratio
`knapsack_brute`	Brute-force verifier for small inputs ($n \leq 20$)
`run_test_case`	Runs all algorithms on a named small test case

4. Full Working Code

See knapsack_dp.py for the complete implementation.

4.1. Core DP Loop

```
for i in range(1, n + 1):
    c = costs[i - 1]        # weight of item i
    r = returns[i - 1]      # value  of item i
    for w in range(W + 1):
        skip = dp[i - 1][w]
        take = (dp[i - 1][w - c] + r) if c <= w else 0
        dp[i][w] = max(skip, take)
```

4.2. Space-Optimised 1D DP

```
dp = [0.0] * (W + 1)
for i in range(n):
    for w in range(W, costs[i] - 1, -1):   # RIGHT-TO-LEFT is critical
        dp[w] = max(dp[w], dp[w - costs[i]] + returns[i])
```

Right-to-left traversal ensures each item is counted at most once (0/1 constraint).

5. Experimentation & Results

5.1. Test Configuration

```
Number of investments: 120 (tiered, seed=7, globally known names)
Budget (W)           : $500K
```

5.2. Named Test Cases - All Brute-Force Verified

Screenshot 1 → Classic 5-item Textbook DP = 35.00 | Greedy = 28.00 | Gap = **20% suboptimal**

Screenshot 2 → Greedy Failure Demo DP picks 1 large item → return = 16 Greedy picks 5 small items → return = 15 Gap = **6.25% suboptimal**

Screenshot 3 → Tight Budget & Single Item Both algorithms agree. Edge cases handled correctly.

Screenshot 4 → Choose Better & No Items Fit Algorithm picks higher-return item when costs are equal. Returns 0 gracefully when nothing fits - no crash.

Screenshot 5 → Large Greedy Gap transition to Full Dataset Final test case passes. Full 120-item dataset loaded successfully.

5.3. Full Dataset Results

```
═══════════════════════════════════════════════════════════
  DYNAMIC PROGRAMMING RESULT
═══════════════════════════════════════════════════════════
  Maximum Return  :   $2,633.17K
  Total Cost Used :     $500K      (full budget used)
  Items Selected  :     37

  Top picks by return:
  Toyota Bond        Cost=$20K   Return=$139.38K  Ratio=6.97
  Los Angeles House  Cost=$23K   Return=$134.59K  Ratio=5.85
  Copper             Cost=$22K   Return=$130.08K  Ratio=5.91
  Balanced Fund      Cost=$31K   Return=$127.98K  Ratio=4.13
  Visa               Cost=$23K   Return=$124.98K  Ratio=5.43
═══════════════════════════════════════════════════════════

  DP (2D) Runtime : 11.989 ms
  DP (1D) Runtime :  6.771 ms  [✓ MATCH — same answer, faster]
```

6. Greedy Approach Comparison

6.1. Why Greedy Fails for 0/1 Knapsack

Greedy commits to high-ratio items early. When those are cheap, they fill the budget fast and leave no room for fewer but higher-return items.

Proof from Test Case 2:

Item	Cost	Return	Ratio
A–E	1 ea	3 ea	3.00
F	6	16	2.67

Budget = 6. Greedy picks A+B+C+D+E = 15. DP picks F = **16**. Greedy loses.

6.2. Full Dataset Comparison

```
═══════════════════════════════════════════════════════════
  DP vs GREEDY - FINAL COMPARISON
═══════════════════════════════════════════════════════════
  Metric                          DP          Greedy
  Max Return ($K)           2,633.17        2,633.17
  Total Cost Used ($K)           500             500
  Items Selected                  37              37
  Greedy Suboptimality         0.00%

  Verdict: Both found same value
═══════════════════════════════════════════════════════════
```

Both found the same value on this dataset. However, the test cases prove greedy can be **6–20% suboptimal** on other inputs. DP is always guaranteed optimal — greedy is not.

7. Complexity Analysis

7.1. Time Complexity

Phase	Complexity	Actual Count
Load CSV	O(n)	120 rows
DP Fill	O(n × W)	60,000 operations
Traceback	O(n)	120 steps
Greedy	O(n log n)	~720 comparisons

7.2. Space Complexity

Approach	Space	Actual Size
2D DP table	O(n × W)	60,000 float cells
1D DP array	O(W)	500 float cells (120× smaller)

7.3. Measured Runtimes (actual run)

Algorithm	Runtime
DP (2D)	11.989 ms

DP (1D)	6.771 ms
Greedy	< 1 ms

8. Screenshots Summary

All screenshots are located in the `screenshoots/` folder.

#	File Name	What It Shows
1	Screenshot_01_Test_Cases_DP_vs_Greedy.jpg	Classic test — DP=35, Greedy=28 (20% gap)
2	Screenshot_02_Greedy_Failure_and_Edge_Case.jpg	Greedy failure proof + all-items-fit edge case
3	Screenshot_03_Tight_Budget_and_Single_Item.jpg	Boundary conditions handled correctly
4	Screenshot_04_Choose_Better_and_No_Items_Fit.jpg	Equal-cost choice + zero-result edge case
5	Screenshot_05_Last_TestCase_and_DP_Result_Summary.jpg	Test cases complete + full dataset begins
6	Screenshot_06_DP_Optimal_Portfolio_Full_List.jpg	All 37 DP-selected investments
7	Screenshot_07_Runtime_and_Greedy_Summary.jpg	Runtime comparison + 1D DP verification
8	Screenshot_08_Greedy_Full_Portfolio_List.jpg	All 37 Greedy-selected investments
9	Screenshot_09_Final_Comparison_and_Complexity.jpg	DP vs Greedy table + complexity summary

9. Improvements & Extensions

Extension	Description
Risk Constraint	Add a second weight dimension for risk score
Sector Diversification	At most k investments per sector
Multi-Period Horizon	Different investments mature at different times
Liquidity Constraint	Add a liquidity weight alongside cost
Integer Programming	Use PuLP or Gurobi for large-scale real portfolios

10. Conclusion

1. **DP guarantees optimality** -verified against brute-force on all 8 test cases
2. **Greedy is fast but unreliable** - proven 6–20% suboptimal on crafted inputs
3. **Space optimisation works** - 1D DP uses 120× less memory, same answer, faster runtime
4. **Real-world names make results readable** - globally known brands (Apple, Bitcoin, Toyota Bond) make the portfolio output immediately understandable

11. References

1. Cormen et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Ch.15 (DP), Ch.16 (Greedy)
2. Kellerer, Pferschy & Pisinger (2004). *Knapsack Problems*. Springer.
3. GeeksforGeeks. *0-1 Knapsack Problem DP-10*. https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/
4. Dataset: Randomly generated using `generate_dataset.py` (tiered, seed=7)
5. Python 3 Documentation. https://docs.python.org